

三、修 Bug 实战剖析：多轮纠错与防死循环

核心问题：Agent 收到报错信息后，如何决定下一步行动？

OODA 定位：Orient 环节（从失败中学习）/ 完整走一遍 L→R→V 循环

从上一节的问题切入

上一节结尾留了一个问题：

当 Agent 在 Localization → Repair → Validation 循环中遇到失败——比如改了代码但测试还是报错——它如何从失败中学习而不是陷入死循环？

上一节讲了 Act（工具调用机制），Agent 有了“手”可以操作外部世界。但光有手不够——当操作结果不符合预期时，Agent 需要理解发生了什么，然后做出正确判断。

这就是 OODA 循环中的 **Orient**（定向/理解）。本节的核心论点：

Orient 的质量决定了 Agent 是能纠错，还是陷入死循环。

3.1 关键时刻：测试报错后的那条 Thought

一个真实的修 Bug 流程

〔Demo 3 • Live〕用 MVP + Qwen3-30B-A3B 修复 `buggy_calc.py` ——一个成绩统计函数，包含两个 bug。

```
# buggy_calc.py - 成绩统计函数
def stats_report(scores):
    # Bug 1: 除数错误 - 应该除以 len(scores)，却除以 len(scores) - 1
    avg = sum(scores) / (len(scores) - 1)

    # Bug 2: 边界错误 - 及格线是 >= 60，却写成了 > 60
    passing = [s for s in scores if s > 60]
    pass_rate = len(passing) / len(scores) * 100

    return {"average": round(avg, 1), "pass_rate": round(pass_rate, 1), ...}
```

测试数据： `scores = [80, 60, 90, 70, 50]` - 正确 average: 70.0 (350/5) , bug 给出 87.5 (350/4) - 正确 pass_rate: 80.0% (4 人 ≥ 60) , bug 给出 60.0% (3 人 > 60)

演示指令（直接复制）

先运行 `pytest test_buggy_calc.py -v` 看报错，然后读 `buggy_calc.py` 找到 bug 并用 Edit 修复。每修一个 bug 就跑一次 `pytest` 验证。不要问我，直接修。

演示前：确保 `buggy_calc.py` 是原始带 bug 的版本（`git checkout mvp/tests/buggy_calc.py`）。

演示中：agent 的输出是流式的，每轮会打印 `— Round N/10 — + □Thought + □Action + □Result`，边跑边标注当前处于 L/R/V 的哪个阶段。

每步标注 L/R/V 和 OODA

Round 1 – Localization / Observe

Agent: Read("buggy_calc.py") → 看到代码全貌

Agent: Bash("pytest test_buggy_calc.py -v")

→ 3 failed: test_average (87.5 $\neq$ 70.0), test_pass_rate (60.0 $\neq$ 80.0), test_all_pass

Round 2 – 【关键时刻：Orient】

Agent 看到了测试输出：

"AssertionError: Expected average 70.0, got 87.5"

"AssertionError: Expected pass_rate 80.0%, got 60.0%"

▼ 这条 Thought 决定了一切 ▼

Agent 的推理（好的 Orient）：

"87.5 = 350/4, 但应该是 350/5。除数是 len-1=4 而非 len=5。

这是 Bug 1。让我先修这个。"

Round 3 – Repair / Act

Agent: Edit(old="len(scores) - 1", new="len(scores)")

Round 4 – Validation / Observe（新一轮）

Agent: Bash("pytest test_buggy_calc.py -v")

→ test_average PASSED ✓

→ test_pass_rate STILL FAILED: "Expected 80.0%, got 60.0%"

Round 5 – 【第二个 Orient 时刻】

Agent 看到了新的错误信息（与上次不同！）：

"pass_rate 60.0% 意味着只有 3 人通过。期望 80% 是 4 人。

60 分的学生没被算进去。> 60 应该改成 ≥ 60 。"

这就是多轮纠错的核心：每次 Validation 失败产生新的 Observation，模型需要从中提取新信息（Orient），而不是重复上一次的修复。

Round 6 – Repair + Validation

Agent: Edit(old="s > 60", new="s >= 60")

Agent: Bash("pytest test_buggy_calc.py -v") → 6 passed ✓ 全部通过

要点：Orient 是什么

在 OODA 循环中，Orient 不是简单的“看到了错误”——那是 Observe。Orient 是**理解错误的含义并形成假设**：

Observe:	"test_average: Expected 70.0, got 87.5"	← 原始信号
Orient:	"87.5 = 350/4。除数是 4 不是 5。len-1 是错的。"	← 推理 + 假设
Decide:	"把 len(scores)-1 改成 len(scores)"	← 行动方案
Act:	Edit(...)	← 执行

差的 Orient 长什么样？

Observe:	"test_average: Expected 70.0, got 87.5"	
Orient:	"测试失败了。让我再试一次。"	← 没有理解为什么
Decide:	"把 sum(scores) 改成别的什么"	← 盲目修改

这条低质量的 Thought 就是死循环的起点——Agent 没有从错误信息中提取有价值的信息。

3.2 Reflexion：用自然语言替代梯度

核心创新

〔Reflexion：化失败为经验〕

上面的 Demo 展示了 Agent 在**一次会话内**从失败中学习。但如果 Agent 需要**多次尝试**才能完成任务呢？每次失败的经验能否保留下来？

这就是 Reflexion（Shinn et al., 2023）要解决的问题。它的核心创新用一句话概括：

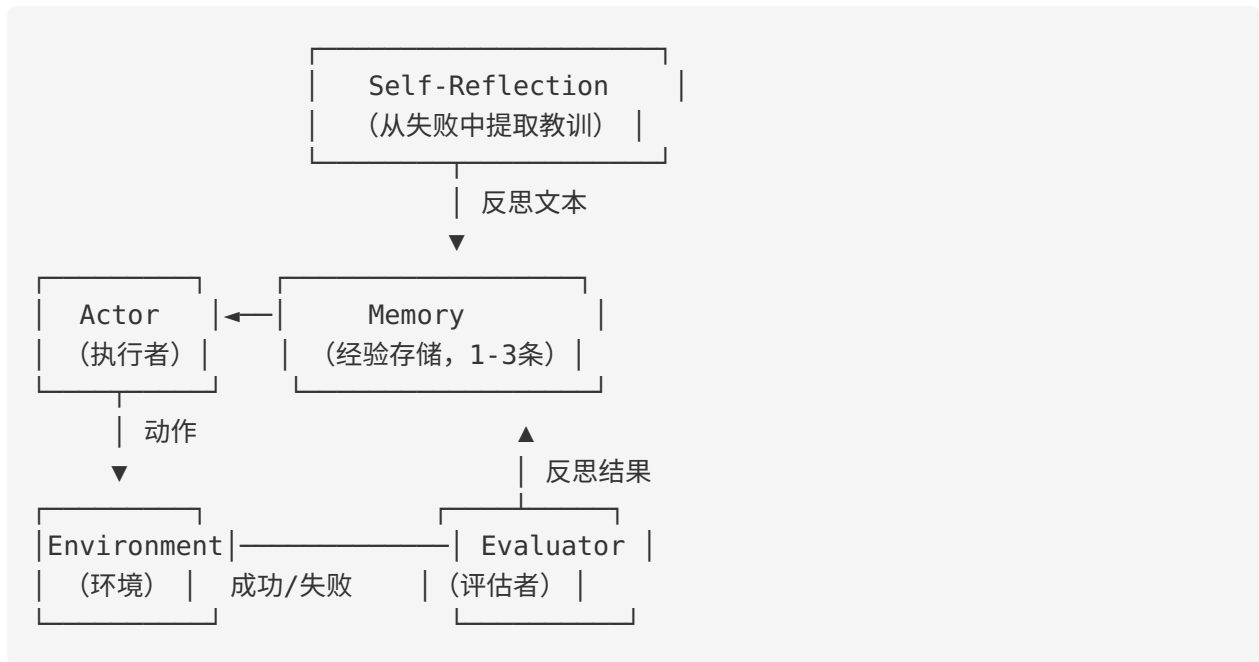
用自然语言的经验总结替代梯度更新——不改模型权重，只改提示词中的记忆。

对比传统强化学习：

传统 RL: 动作 → 奖励信号 → 梯度更新 → 模型参数改变 (需要训练)
Reflexion: 动作 → 评估结果 → 自然语言反思 → 存入记忆 → 下次尝试参考 (只需推理)

三个组件

Reflexion 的架构包含三个协作模块 (论文 Figure 2) :



1. **Actor (执行者)** : 基于当前状态和历史经验生成动作。相当于我们的 Agent 主循环
2. **Evaluator (评估者)** : 判断执行结果是否成功。在编程任务中, 这就是**运行测试**
3. **Self-Reflection (自我反思)** : 失败时不只是说“失败了”, 而是生成一段**自然语言的经验总结**

关键设计: Memory 限制为最多 1-3 条经验 (论文 Section 3, page 5: “we bound mem by a maximum number of stored experiences, Ω , usually set to 1-3”), 避免上下文过长。

实验数据

Reflexion 论文在编程任务上的结果 (Table 1, page 7) :

基准测试	GPT-4 基线 (pass@1)	+ Reflexion	提升
HumanEval (Python)	80.1%	91.0%	+10.9%
HumanEval (Rust)	60.0%	68.0%	+8.0%

基准测试	GPT-4 基线 (pass@1)	+ Reflexion	提升
MBPP (Rust)	71.0%	75.4%	+4.4%

仅靠“记住之前哪里错了”，HumanEval 就从 80.1% 提升到 91.0%。不需要微调、不需要额外训练数据，只需要自然语言形式的失败经验。

论文的消息实验 (Table 3, page 8) 更有说服力：

配置	测试生成	自我反思	Pass@1
基线（无 Reflexion）	无	无	60%
去掉自我反思	有	无	60%（没有提升！）
去掉测试生成	无	有	52%（反而下降）
完整 Reflexion	有	有	68%

两个关键发现： 1. 没有自我反思，光有测试不会提升——说明反思（Orient 环节）才是关键 2. 没有测试，光靠自我反思反而有害——说明 Evaluator 必须有可靠的外部信号

映射到 OODA：Observe（测试信号）和 Orient（反思理解）缺一不可。

为什么编程任务上 Reflexion 特别有效？

1. **评估信号精确：**测试通过/失败是二元的，测试输出（expected X, got Y）精确定位问题
2. **错误可自然语言描述：**“忘了处理空列表”、“off-by-one 错误”——这些经验表述直接可用
3. **经验具有迁移性：**一个任务中“注意边界条件”的教训，对类似任务同样有效

Reflexion 在 Claude Code 中的映射

CC 没有显式实现 Reflexion 的三组件架构，但思想在两个层面自然存在：

隐式 Reflexion — 对话上下文即 Memory

在一次会话中，每次修复失败的完整记录（tool_use + error result）留在对话历史中。模型在下一轮推理时可以“看到”之前所有的失败经历：

```
[Round 3] Edit: 把 range(len(n)) 改成 range(len(n)-1)
[Round 4] Bash: pytest → "Sum: 10, expected 15" ← 失败记录留在上下文中
```

```
[Round 5] Thought: "上次的修改方向错了。range 不是问题，
                    index 才是。应该改 numbers[i+1] → numbers[i]"
```

对话上下文扮演了 Reflexion 中 Memory 的角色。**Agent 没有显式的 Self-Reflection 步骤，但 Thought 环节隐式完成了“从失败中提取教训”的工作。**

显式 Reflexion — CLAUDE.md 作为持久化记忆

用户可以在 `CLAUDE.md` 中记录经验教训，这些内容在每次会话开始时加载：

```
# CLAUDE.md
## Known Issues
- 修改 auth 模块时，一定要同步更新 middleware 中的 token 验证
- 数据库迁移前先用 pg_dump 备份，不要用自定义脚本
```

CC 源码确认 5 层加载顺序（`claudemd.ts`）：`managed` → `user` → `project` → `local` → `autoMem`。这本质上是**人工 Reflexion**——人类将失败经验编码为自然语言，持久化存储，供 Agent 在后续会话中参考。

3.3 上下文压缩：模型如何管理自己的记忆

问题：上下文窗口是有限的

上一节说“对话上下文 = Reflexion 的 Memory”。但这个 Memory 有一个物理限制：**模型的上下文窗口**。

一个修 Bug 的会话可能 5 轮就结束。但一个复杂的开发任务可能需要 50 轮甚至更多——每轮的 `tool_use`、`tool_result` 都在消耗上下文空间。当对话超过窗口上限时会发生什么？

- 最朴素的做法是**截断**：丢弃最早的消息。但那些消息可能包含关键的失败经验
- CC 的做法更精细：**让模型自己决定什么值得记住**

模型在幕后做的工作

CC 的上下文压缩不是简单的文本截断，而是**让模型给自己写摘要**。这里的“模型”就是同一个 LLM——做编码工作的是它，做记忆压缩的也是它。

当对话接近窗口上限时（CC 源码 `autoCompact.ts`：阈值 = 上下文窗口 - 20K 预留输出 - 13K 缓冲），CC 触发 `auto-compact`：

触发条件：当前对话 token 数 > contextWindow - 33K

压缩流程：

1. 将整段对话作为输入
2. 加上一个 compact prompt（指导如何压缩）
3. 模型输出一份结构化摘要
4. 这份摘要替换掉原始对话，成为后续推理的唯一记忆

Compact Prompt：一个精心设计的元提示

compact prompt（CC 源码 `compact/prompt.ts`，行 61-143）本质上是一个**元提示**——它不是让模型写代码或回答问题，而是让模型**对自己的工作历史做结构化提取**。

prompt 的核心指令：

“Your task is to create a detailed summary of the conversation so far, paying close attention to the user’s explicit requests and your previous actions.”

模型被要求输出的摘要必须包含 **9 个固定段落**：

段落	内容	与 Reflexion 的关系
1. Primary Request and Intent	用户要求做什么	保持任务目标不变
2. Key Technical Concepts	涉及的技术概念	Orient 所需的领域知识
3. Files and Code Sections	读过/改过哪些文件	Observe 的记录
4. Errors and fixes	遇到了什么错误、怎么修的	Reflexion Memory
5. Problem Solving	解决了什么问题	成功经验
6. All user messages	用户的所有消息	用户反馈（人在回路）
7. Pending Tasks	待办事项	未完成的工作
8. Current Work	当前正在做什么	下一步 Orient 的起点
9. Optional Next Step	建议的下一步	行动方向

注意第 4 段——“Errors and fixes”。这就是压缩版的 Reflexion Memory：确保所有失败经验和修复方案在压缩后依然保留。compact prompt 中对这一段的原文要求：

“List all errors that you ran into, and how you fixed them. Pay special attention to specific user feedback that you received, especially if the user told you to do something differently.”

这不是巧合，这是设计——压缩系统的设计者知道，失败经验是最不应该被丢弃的信息。

模型的两阶段压缩过程

CC 的 compact prompt 还有一个巧妙设计：模型在输出最终摘要之前，先在 `<analysis>` 标签中做一轮**推理草稿**（行 31-44）：

[Phase 1: 推理草稿 – `<analysis>` 标签]

"让我按时间线过一遍对话。Round 1 读了文件，Round 2 发现了 bug，Round 3 修了但测试还是失败——原来 `> 60` 应该是 `>= 60`。Round 4 修了第二个 bug，测试全过了。关键教训：这个文件有两个独立的 bug，不能只看第一个错误就下结论。"

[Phase 2: 结构化摘要 – `<summary>` 标签]

1. Primary Request: 修复 `buggy_calc.py` 中的统计计算错误
4. Errors and fixes: Bug 1 (`avg` 除数 `len-1`→`len`)，Bug 2 (及格线 `>60`→`>=60`)
8. Current Work: 两个 bug 均已修复，测试全部通过
- ...

推理草稿随后被丢弃（`formatCompactSummary` 函数用正则剥离 `<analysis>` 块），只保留结构化摘要。这和 CoT（Chain of Thought）的原理一致：**让模型“想”一遍再输出，比直接输出更准确**——即使在做压缩这种“元任务”上也是如此。

原理层面：这本质上是什么

从模型原理角度看，auto-compact 是一个**信息瓶颈**（Information Bottleneck）操作：

输入：完整对话历史（可能 100K+ tokens）

↓

模型做有损压缩

↓

输出：结构化摘要（~5K-10K tokens）

几个关键特性：

1. **有损且语义感知**：不是按位置截断（那是无损但无意义的），而是模型根据语义重要性决定保留什么。第 4 段“Errors and fixes”的优先级被 prompt 显式提高

- 2. **模型给未来的自己写信**：压缩后的摘要当前模型实例留给下一个实例的“认知遗产”——两个实例之间唯一的桥梁就是这份摘要。摘要的质量直接决定后续推理的质量
- 3. **CoT 提升压缩质量**：`<analysis>` 草稿让模型先“理清思路”再压缩，与 CoT 在推理任务上的提升机制完全一致——即使在“元任务”（自我总结）上也有效
- 4. **任务感知的 schema**：9 个固定段落不是通用摘要模板，而是针对编程 Agent 任务精心设计的。如果是客服 Agent，段落会完全不同

类比人类认知：- 上下文窗口 = 工作记忆（容量有限，约 7 ± 2 个 chunk）- auto-compact = 程序员在长时间调试后“停下来理理思路”，在本子上记下关键发现 - 9 段结构 = 笔记模板（不是流水账，而是按“什么出了错、怎么修的、下一步做什么”分类）

CC 的六层压缩策略

auto-compact 只是 CC 六层压缩中的一层。完整策略从轻到重（CC 源码 `query.ts` + `autoCompact.ts` + `compact.ts`）：

层级	机制	触发条件	原理
1	Tool result budget	每次工具返回	截断过长的工具输出（机械截断）
2	Snip compact	feature flag	历史裁剪
3	Microcompact	对话增长时	清除旧的 tool result 内容体（保留结构）
4	Context collapse	特定条件	投射压缩视图
5	Auto-compact	token > 阈值	模型做全对话摘要（本节重点）
6	Reactive compact	API 返回 prompt-too-long	紧急压缩，最后的逃生舱

设计哲学是渐进式降级：先用轻量、无损的方式节省空间（层 1-4），只有当这些都不够时才触发昂贵的模型摘要（层 5），而层 6 是极端情况下的紧急逃生——宁可有损也不能让会话崩溃。

一个工程细节值得注意：auto-compact 有熔断器（`MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3`）。CC 源码注释记录了为什么需要它：“1,279 sessions had 50+ consecutive failures (up to 3,272) in a single session,

wasting ~250K API calls/day globally”——没有熔断器的压缩系统本身可能成为无限循环的源头。

压缩后的信息恢复

auto-compact 之后，CC 还会做一步**选择性恢复**（compact.ts）：

- 重新注入最近读过的 5 个文件（最多 50K tokens）
- 重新注入活跃的 skill 内容（最多 25K tokens）
- 跳过已经在摘要中出现的文件（避免重复）

这相当于：压缩是“概括性记忆”，恢复是“把最可能需要的细节放回手边”。两者配合确保模型既有全局视角（摘要），又有局部细节（最近文件）。

与 Reflexion 的连接

回到 Reflexion 的框架：

- Reflexion 论文的 Memory：字符串列表，最多 1-3 条，简单追加
- CC 的隐式 Memory：对话上下文，通过 6 层压缩维持可用性
- 两者的共同挑战：**在有限空间内保留对未来决策最有价值的信息**

Reflexion 论文选择了“限制条数”（ $\Omega = 1-3$ ），简单但粗暴——老的经验直接被新的替换。CC 选择了“模型自主压缩”，更精细但依赖模型的判断力。

这背后是同一个 **trade-off**：记忆太少会遗忘重要教训（死循环风险），记忆太多会稀释关键信息（信噪比下降）。Reflexion 用硬限制解决，CC 用模型智能解决。

3.4 当 Orient 失灵：死循环的诊断与 Harness 兜底

死循环的症状

当 Orient 环节失灵时，Agent 的行为表现出可识别的模式：

```
Round 5: Edit(old="len-1", new="len")
Round 6: Bash("pytest") → FAIL: "Expected 80.0, got 60.0"
Round 7: Edit(old="len", new="len-1")           ← 撤回了上一步的修改！
Round 8: Bash("pytest") → FAIL: "Expected 70.0, got 87.5"   ← 回到原始错误
Round 9: Edit(old="len-1", new="len")           ← 又改回来...
Round 10: max_rounds reached. Stopping.
```

这是典型的**振荡模式**——Agent 在两个状态之间来回切换，无法前进。

三种 Orient 失灵的原因

1. **模型推理能力不足** Orient 本身质量差——模型看到了 `Expected 80.0, got 60.0` 但无法推理出“是 $\geq$ 和 $>$ 的边界问题”。这是小模型（如 3B 激活参数）的常见瓶颈。
2. **关键信息被挤出上下文** 在长会话中，早期的失败经验可能已经被压缩丢弃或被大量新信息稀释。模型“忘了”之前已经尝试过某个方案失败了，于是重复尝试。
3. **上下文信噪比过低** 对话中充满了重复的失败尝试和冗长的 tool result，关键的错误分析被淹没。模型的 attention 分散在大量低价值信息上，无法聚焦到真正重要的 Observation。

这三种原因分别对应三种工程应对：更强的模型（根本解）、更好的压缩（保留关键信息）、更精简的工具输出（提高信噪比）。

Harness 的三重兜底

CC 源码（`query.ts`，1400+ 行）中的恢复机制不是“一个 circuit breaker”那么简单，而是一套分层防线：

第一层：信息质量保障（预防） - Tool result budget：截断过长的工具输出，确保每轮 Observation 精简有价值 - 上下文压缩（6 层策略）：确保关键失败经验不被丢弃 - CC 源码 `compact/prompt.ts` 第 4 段“Errors and fixes”被显式优先保留

第二层：行为检测与纠正（干预） - **Nudge 机制**：当模型 `end_turn` 但任务未完成时，注入引导消息（“你还没完成，继续”） - **max_output_tokens 恢复**：模型输出被截断时，CC 注入一条精确的恢复指令。CC 源码中的原文（`query.ts` 行 1230-1236）：

“Output token limit hit. Resume directly – no apology, no recap of what you were doing. Pick up mid-thought if that is where the cut happened. Break remaining work into smaller pieces.”

这条消息的每个词都是有意设计的：“**no apology**”防止模型浪费 token 说“抱歉我被截断了”；“**no recap**”防止模型复述刚才的内容（最常见的 token 浪费）；“**pick up mid-thought**”允许从句子中间继续；“**break into smaller pieces**”引导模型调整策略避免再次被截断。最多重试 3 次（`MAX_MAX_OUTPUT_TOKEN_RECOVERY = 3`），第一次截断时还会升级到 64K `max_output_tokens`（`ESCALATED_MAX_TOKENS`） - **MVP 对应**：我们 `client.py` 的 nudge 机制（检测模型“假装调工具”但没发出 `tool_call`）

第三层：强制终止（止损） - **max_turns**：硬性上限，超过就停。CC 的 12 个 return 退出点中，`max_turns` 是最后的安全阀 - **Reactive compact**：API 返回 prompt-too-long 时的紧急压缩——不能让会话崩溃 - **熔断器**：auto-compact 连续失败 3 次后停止重试，防止压缩系统自身陷入循环

核心认知

好的 Agent ≠ 永远不犯错
好的 Agent = 有足够的 Orient 能力从错误中学习

- + 有足够好的记忆管理保留失败经验
- + 有 Harness 兜底确保即使 Orient 失灵也不会无限消耗

〔Demo 3 后半 • Live〕如果小模型在修 Bug 过程中出错——不要重启。当场标注“这是 Orient 失灵”，指出模型的 Thought 缺少了什么信息，或者做了什么错误推理。这就是小模型作为教学工具的价值：它的失败模式清晰可见、可解释。

3.5 本节小结：从失败中学习的完整链路

回到二维认知地图，本节覆盖了 Orient 环节：

	Localization	Repair	Validation
Observe	(下一节)		
Orient (本节)	★ 测试报错后的推理 ★ 上下文压缩	★ Reflexion 经验积累	★ 从 V 失败中学习 ★ 死循环诊断

三个核心收获：

- Orient 质量决定成败**：Agent 看到错误信息后的那条 Thought，决定了它是走向修复还是走向死循环。好的 Orient 从 `Expected X, got Y` 中提取出因果推理（“ $87.5 = 350/4$ ，除数错了”），差的 Orient 只是“让我再试一次”。
- Reflexion = 语言替代梯度**：不改权重，只用自然语言总结失败经验并保留在记忆中。HumanEval 80.1% → 91.0%（Reflexion 论文 Table 1）。在 CC 中，对话上下文是隐式 Memory，CLAUDE.md 是显式 Memory。

3. **上下文压缩是 Reflexion 的基础设施**：没有压缩，长任务中的失败经验会被挤出上下文。CC 的 auto-compact 让模型自己做有损压缩——按 9 个结构化段落提取，其中“Errors and fixes”确保失败经验优先保留。这是模型在“编码”之外的另一项关键工作：**管理自己的记忆**。

下一节的问题是：**Agent 在 Localization 阶段如何高效感知代码库？在百万行代码中，如何用“刚好够”的信息量找到目标，而不是把整个文件塞进上下文？**